

zh3r0 2020 Crypto - Analyze Me

deuterium — 2020-06-18

Canonical HTML: [https://deut-erium.github.io/WriteUps/2020/zh3r0/crypto/Analyze me/2020-06-18-zh3r0-2020-Analyze-Me.html](https://deut-erium.github.io/WriteUps/2020/zh3r0/crypto/Analyze%20me/2020-06-18-zh3r0-2020-Analyze-Me.html)

Abstract

CTF writeup: zh3r0 CTF 2020's Analyze Me can't pick a base. Decode the mixed-base chunks, then undo XOR and repeated byte substitutions in the second act.

Analyse Me

Description

```
i have a server's ip which gives us the flag,  
but i'm not so good in cracking it. Can you decode the flag for me?  
nc crypto.zh3r0.ml 3871
```

Author : Whit3_D3vil

Challenge files

- [server.py](#) earlier [analyse.py](#) which is quite unsolvable according to me

Lets take a look at contents of [server.py](#)

```
from Crypto.Util.number import *  
from Crypto.Util.strxor import strxor  
from binascii import *  
from base64 import *  
import random  
from flags import *  
import string  
  
assert len(msg)==80  
  
msg1=msg  
def from_the_bases(msg,count):  
    count=(count%4)+1  
    msg=''.join(msg)  
    if count == 1:  
        return b64encode(msg.encode())  
    elif count == 2:  
        return b32encode(msg.encode())  
    elif count == 3 :  
        return b85encode(msg.encode())  
    else:  
        return hexlify(msg.encode())
```

```

print('''Hello!
Welcome to Xor analysis..
There are two parts.
All the best ;)

Here is the first part:

''')
count=0
for i in range(0,len(msg1),4):

    print(bytes_to_long(from_the_bases(msg1[i:i+4],count)),end='|')
    count+=1

print()

user_input=input('Enter the decoded message:')

assert len(user_input)==80

if user_input==msg:
    print('You have done well.')
    print('Here is the key: ',bytes_to_long(key.encode()))

else:
    print('Try harder ;)')
    quit()

print('\nYou completed the first part \n Here is the second and final part ;) ')

print('GOOD LUCK DECODING!!!! \n')

table={'0':
{'0':'63','1':'7c','2':'77','3':'7b','4':'f2','5':'6b','6':'6f','7':'c5','8':'30','9':
:'01','a':'67','b':'2b','c':'fe','d':'d7','e':'ab','f':'76'},
'1':
{'0':'ca','1':'82','2':'c9','3':'7d','4':'fa','5':'59','6':'47','7':'f0','8':'ad','9':
:'d4','a':'a2','b':'af','c':'9c','d':'a4','e':'72','f':'c0'},
'2':
{'0':'b7','1':'fd','2':'93','3':'26','4':'36','5':'3f','6':'f7','7':'cc','8':'34','9':
:'a5','a':'e5','b':'f1','c':'71','d':'d8','e':'31','f':'15'},
'3':
{'0':'04','1':'c7','2':'23','3':'c3','4':'18','5':'96','6':'05','7':'9a','8':'07','9':
:'12','a':'80','b':'e2','c':'eb','d':'27','e':'b2','f':'75'},
'4':
{'0':'09','1':'83','2':'2c','3':'1a','4':'1b','5':'6e','6':'5a','7':'a0','8':'52','9':
:'3b','a':'d6','b':'b3','c':'29','d':'e3','e':'2f','f':'84'},
'5':
{'0':'53','1':'d1','2':'00','3':'ed','4':'20','5':'fc','6':'b1','7':'5b','8':'6a','9':
:'cb','a':'be','b':'39','c':'4a','d':'4c','e':'58','f':'cf'},
'6':
{'0':'d0','1':'ef','2':'aa','3':'fb','4':'43','5':'4d','6':'33','7':'85','8':'45','9':
:'f9','a':'02','b':'7f','c':'50','d':'3c','e':'9f','f':'a8'},
'7':
{'0':'51','1':'a3','2':'40','3':'8f','4':'92','5':'9d','6':'38','7':'f5','8':'bc','9':
:'b6','a':'da','b':'21','c':'10','d':'ff','e':'f3','f':'d2'},
'8':
{'0':'cd','1':'0c','2':'13','3':'ec','4':'5f','5':'97','6':'44','7':'17','8':'c4','9':

```

```

:a7','a':'7e','b':'3d','c':'64','d':'5d','e':'19','f':'73'},
  'g':
{'0':'60','1':'81','2':'4f','3':'dc','4':'22','5':'2a','6':'90','7':'88','8':'46','9'
:'ee','a':'b8','b':'14','c':'de','d':'5e','e':'0b','f':'db'},
  'a':
{'0':'e0','1':'32','2':'3a','3':'0a','4':'49','5':'06','6':'24','7':'5c','8':'c2','9'
:'d3','a':'ac','b':'62','c':'91','d':'95','e':'e4','f':'79'},
  'b':
{'0':'e7','1':'c8','2':'37','3':'6d','4':'8d','5':'d5','6':'4e','7':'a9','8':'6c','9'
:'56','a':'f4','b':'ea','c':'65','d':'7a','e':'ae','f':'08'},
  'c':
{'0':'ba','1':'78','2':'25','3':'2e','4':'1c','5':'a6','6':'b4','7':'c6','8':'e8','9'
:'dd','a':'74','b':'1f','c':'4b','d':'bd','e':'8b','f':'8a'},
  'd':
{'0':'70','1':'3e','2':'b5','3':'66','4':'48','5':'03','6':'f6','7':'0e','8':'61','9'
:'35','a':'57','b':'b9','c':'86','d':'c1','e':'1d','f':'9e'},
  'e':
{'0':'e1','1':'f8','2':'98','3':'11','4':'69','5':'d9','6':'8e','7':'94','8':'9b','9'
:'1e','a':'87','b':'e9','c':'ce','d':'55','e':'28','f':'df'},
  'f':
{'0':'8c','1':'a1','2':'89','3':'0d','4':'bf','5':'36','6':'42','7':'68','8':'41','9'
:'99','a':'2d','b':'0f','c':'b0','d':'54','e':'bb','f':'16'}}

flag=final_flag#final_flag is a variable from my flags module

final_key=key#key is a variable from my flags module, ohhh you have the key if you
solved the first part

list_flag=[hexlify(flag[i:i+4].encode()).decode() for i in range(0,len(flag),4) ]

final_list=[]

rand_num=random.randint(2,4)

for c in list_flag:
    chr=c
    num=0
    while num < rand_num:
        chr=table[chr[0]][chr[1]]+table[chr[2]][chr[3]]+table[chr[4]]
[chr[5]]+table[chr[6]][chr[7]]
        num+=1
    final_list.append(chr)

key_list=[hexlify(final_key[i:i+4].encode()).decode() for i in range(0,len(key),4)]

xor_list=[strxor(i.encode(),j.encode()) for i,j in zip(final_list,key_list)]

print(xor_list)

user_input=input('Enter the flag:')

if user_input==final_flag:
    print('Perfect you got the flag ;) \nGo submit it!!!')

else:
    print('You must try harder!!!!!!!')
    quit()

```

Here we have two parts of the challenge, Only after solving the first, we can move forward to the second part.

```
nc crypto.zh3r0.ml 3871
Hello!
Welcome to Xor analysis..
There are two parts.
All the best ;)
```

Here is the first part:

```
5943134639005711677|5491378081737038141|366970695973|3833466206172886320|5640277313745
009981|5351739078059639101|302416945480|3762814891798442803|6354696933901548861|513925
8452082510141|305635213400|3688506584576963897|5568232986773634365|5139251786226882877
|357308525154|3847819437120304993|7008813202989464893|5786655223480211773|306693940071
|3689633605503693413|
Enter the decoded message: ^C
```

The first part is simple (figuratively), We are presented a "|" separated list of integers each element of which is bigint representation of ascii string of random base encodings out of bases :- 16,32,64,85

Lets see what we get after processing data from the network and converting it to strings

```
from pwn import remote, xor
from base64 import b32decode, b64decode, b85decode, b16decode
from Crypto.Util.number import long_to_bytes #to convert bigint to bytestring
import binascii #for binascii.Error which is raised when base decoding is not
performed correctly
import string # for printable characters

HOST, PORT = "crypto.zh3r0.ml", 3871

REM = remote(HOST, PORT)

chall = REM.recvuntil(b'Enter the decoded message:')
print(chall.decode())

chall_data = chall.split(b'\n\n\n')[1].split(b'|\n')[0].split(b'|')
chall_data = [long_to_bytes(int(i)) for i in chall_data]
print(chall_data)
```

```
[b'RzAwRA==', b'L5KEQMY=', b'Uq(q%', b'53545f50', b'NFJUXw==', b'JE2V6RA=', b'FitaH',
b'48455233', b'X0k1Xw==', b'GRPUMMI=', b'G)G@X', b'30525f59', b'MFVfSA==',
b'GRJEIXY=', b'S1?jb', b'5f3d5f7a', b'aDNyMA==', b'PNTDI2Y=', b'Ghb#g', b'34677d2e']
```

Seems all right, now we need to decode the stuff.

Well, one could say (most of the times) which decoding is it by simply looking at the characters used **AND** by the size of encoded string.

Encoding	Block size	characters
Base16	2	0-9a-f
Base32	8	0-9A-Z
Base64	4	0-9a-zA-z+/
Base85	1	0-9a-zA-Z!#\$%&()*+,-;=<=>?@^_{ }~\`

But why do these have an association of block size to decode properly?

Because, to decode a string, we need the result should be a byte-string.

and a byte is 8 bits, whereas what each character of an encoding could mean is its size.

Base16 16 values = 4 bits =, each character can contain 4 bits of information. And to make a multiple of 8 bits, we would require 2 characters.

Base32, 32 values = 5bits =, each character can contain 5 bits of information. And to make a multiple of 8 bits, we would require a block of 8 characters, and each block would decode to a byte-string of size 5

Base64, 64 values = 6bits =, we would require 4 characters to form a multiple of 8, and hence each 4 character block decodes to 3 character block

Base85? We have a weird base, we stop caring. 😊

Enough of detour, lets decode now

We will simply start from lower base, try to decode, if it results in error, move to an higher base.

But there's one caveat, the string may actually decode on a lower base, and produce garbage decoding.

We may check for decoded string to be in printable domain and raise an error otherwise.

This way we may ensure correctness.

```

PRINTABLE = string.printable.encode()

def check_valid(string):
    return all(i in PRINTABLE for i in string)

def decode(m):
    res = []
    try:
        val = bytes.fromhex(m.decode())
        if not check_valid(val):
            raise ValueError
        res.append(val)
    except ValueError:
        try:
            val = b32decode(m)
            if not check_valid(val):
                raise ValueError
            res.append(val)
        except (binascii.Error, ValueError):

```

```

        try:
            val = b64decode(m)
            if not check_valid(val):
                raise ValueError
            res.append(val)
        except (binascii.Error, ValueError):
            try:
                val = b85decode(m)
                res.append(val)
            except binascii.Error:
                pass

    return res

part_1 = b''.join( decode(i)[0] for i in chall_data )
print(part_1.decode())

```

G00D_TH3_FIRST_P4RT_I5_D0N3_HER3_I5_4_F14G_F0R_Y0U_H4RD_W0RK=_zh3r0{f4k3_f14g}.

Awesome! We are now presented with the second part of the challenge

```

You have done well.
Here is the key:
14026239025573390827696489373042940414594632101792988894633779432300596571220387741502
8

You completed the first part
Here is the second and final part ;)
GOOD LUCK DECODING!!!

[b'\x01Z\nU\x05R\x06R', b'\x0b\t\x00RV\x04\x0e\x01', b'\x06\x0b\x00W\n\x06\x05W',
b'\x07\x07\x01\\\x0e\x07\x04P', b'T\x01\x06W\x03\x04\x05R', b'\x04QS\x06\x0b\n\r_',
b'W\x0b\x04P\x0cST\r', b'\nUS\x01\x01U\r\x00', b'\x05T\x03\x02\x05\x08\x03\x03']
Enter the flag:

```

Writing so much in this writeup, I can tell, this could have been easily two challenges. But anyways

```

list_flag=[hexlify(flag[i:i+4].encode()).decode() for i in range(0,len(flag),4) ]

final_list=[]
rand_num=random.randint(2,4)

for c in list_flag:
    chr=c
    num=0
    while num < rand_num:
        chr=table[chr[0]][chr[1]]+table[chr[2]][chr[3]]+table[chr[4]]
        [chr[5]]+table[chr[6]][chr[7]]
        num+=1
    final_list.append(chr)

key_list=[hexlify(final_key[i:i+4].encode()).decode() for i in range(0,len(key),4)]

xor_list=[strxor(i.encode(),j.encode()) for i,j in zip(final_list,key_list)]

print(xor_list)

```

Now this is a really obfuscated way of doing things. It is basically

- Replacing each byte with its substituent in `table`
- Repeating this some random number of times from 2-4
- Xoring the resultant string with the key obtained from first part

For reversing this, we will need to reverse the `TABLE` and xor it with the provided key

The `TABLE` itself was weird way of writing a table XD

```
INV_TABLE = {}
for k,v in TABLE.items():
    for key,val in v.items():
        INV_TABLE[val] = k+key

flag_unxored = bytes.fromhex(xor(b''.join(flag_enc),KEY.hex()).encode()).decode())
```

Now we need to do the de-substitution of the `flag_unxored` bytes, until we get the flag

```
for _ in range(4):
    flag = ""
    for i in flag_unxored:
        flag+=INV_TABLE[hex(i)[2:].zfill(2)]
    flag = bytes.fromhex(flag)
    if b'zh3r0' in flag:
        print(flag)
        break
    flag_unxored = flag
```

zh3r0-Y0u'4r3'4'v3ry'G00d'4naly3r!"

| Interactive controls are omitted from this PDF. See the canonical HTML version.